COMP30026
Models of Computation

Cezary Kaliszyk and **William Umboh**

Lecture 23 – Reductions

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Today

- Reductions
- Focus on algorithms, not low level management of states, and tape head
- We will use ‘Turing machine’ and ‘algorithm’ interchangeably
  - E.g. an algorithm decides a language
- For concreteness, we will also provide Python interpretations
  - See also closure properties lecture notes
  - Turing machine : Python function
  - M accepts w : function call M(w) returns True
  - M rejects w : function call M(w) returns False
  - M does not halt on w: function call M(w) does not return

In the Exam

- You will be asked to:
  - Construct reductions
  - Justify correctness
- The exact format is what’s in lecture notes
- No Python necessary

Reductions

Def. Problem A *reduces to* Problem B if the existence of a decider for B implies a decider for A. Write $A \leq B$.

Proof Strategy: construct decider for A by making calls to decider for B

Example Let L be a language. Then, $L^c \leq L$.

Pf. Let M be a decider for L

Decider S for L^c:

On input w:

Run M on w

Accept if M rejects

Reject if M accepts

```
def S(w):
    return not M(w)
```

IMPORTANT: Decider for B is a black box. Cannot assume anything other than it decides B.
Think of it as an external library

Reductions: Applications

algorithm

Def. Problem A *reduces to* Problem B if the existence of a decider for B implies a decider for A. Write $A \leq B$.

algorithm

1. Software reuse
   - To solve A, it suffices to show $A \leq B$ for some B that is already solved
2. Prove undecidability
   - Contrapositive of $A \leq B$: If A is undecidable, then B is undecidable
   - To show B is undecidable, show $A \leq B$ for some A that is known to be undecidable
   - Useful to explain why you can't solve a problem!
   - Saves time from doing the impossible

Today's Reductions

- TM Acceptance $\leq$ Halting Problem
- TM Acceptance $\leq$ TM Emptiness $\leq$ TM.Equivalence
- Conclude: Halting, TM Emptiness and TM Equivalence are undecidable
- Due to space, I will verbalize proofs of correctness. See lecture notes for details

TM Acceptance $\leq$ Halting Problem

Halting Problem

- Given an encoding of a TM M and a string w, decide if M halts on w
- In Python, given source code of a Python function M and string w, decide if function call M(w) returns
- Recognizer for Halting?
  - Simulate M on w.
  - If M accepts or rejects, accept.

Why Do We Care?

- Static code analysis:
  - Analyse programs through their source code, without executing them
  - Guarantee safety in critical settings e.g. medical, nuclear
- TM Equivalence (Given TMs M_1 and M_2, do they accept the same strings?)
  - Is the ChatGPT-optimized version of your code equivalent?
  - Did it silently introduce a backdoor?
- TM Emptiness (Given TM M, does it accept no string?)
  - LLM guardrails
  - Define a program that takes as input a prompt, feeds it to ChatGPT and returns True if ChatGPT outputs a biological weapon

TM Acceptance $\leq$ Halting Problem

- Given an encoding of a TM M and a string w, decide if M halts on w
- Given code of Python function M and string w, decide if function call M(w) returns

Proof.
Let H be decider for Halting Problem
Idea: Can safely simulate M on w
Decider S for TM Acceptance
On input $\langle M, w \rangle$:

Run H on $\langle M, w \rangle$
If H accepts,
Run M on w
Accept if M accepts
Reject if M rejects
Else, reject

Correctness:

- Always halts
- Accepts if M accepts w
- Rejects if M does not accept w

```
def S(M,w):
    if H(M,w):
        return M(w)
    else:
        return False
```

TM Acceptance $\leq$ TM Emptiness

TM Emptiness

- Given an encoding of a TM M, decide if M accepts no strings $L(M) = \emptyset$
- Given code of Python function M, decide if M(w) returns False for every w

Proof.

Let H be decider for TM Emptiness

Decider S for TM Acceptance

Idea: Create new TM T and run H on it

On input $\langle M, w \rangle$:

Define TM T that on input x:

Run M on w

Accept if M accept

Reject if M reject

Run H on T

Accept if H reject

Reject if H accept

Correctness:

- Always halts
- T does not accept any string iff M does not accept w

```
def S(M,w):
    def T(x):
        return M(w)
    return not H(T)
```

TM Emptiness $\leq$ TM Equivalence

TM Equivalence

$L(M_1) = L(M_2)$

- Given encodings of TM M_1 and M_2, decide if they accept the same strings
- Given code of Python functions M_1 and M_2, decide if for every w, M_1(w) returns True if and only if M_2(w) returns True

Proof.

Let H be decider for TM Equivalence

Decider S for TM Emptiness

On input $\langle M \rangle$:

Define TM N that on input x:

Reject

Run H on $\langle M, N \rangle$

Accept if H accepts

Reject if H rejects

Correctness:

- Always halts
- N accepts no string
- M accepts no string iff it is equiv. to N

```
def S(M):
    def N(x):
        return False
    return H(M,N)
```

But wait, there's more!

- TM Acceptance, Halting, TM Emptiness, TM Equivalence undecidable
- These are questions about program behavior
- Rice's Theorem: Every nontrivial property about program behavior is undecidable. 
- Note: Rice's Theorem is unassessed and cannot be used to prove undecidability on exam

Summary

- Reductions $A \leq B$ to prove undecidability
- Idea 1: Create new TM
- Idea 2: A is special case of B
- Exam:
  - If asked to prove undecidability, can reduce from TM Acceptance, Halting, TM Emptiness, TM Equivalence
  - If asked to prove $A \leq B$, then you must reduce from A to B
  - You can also prove $A \leq C$ and $C \leq B$ (advanced)

Halting Problem

Another way to see why we cannot have a decider for the Halting problem is that we can use it to turn recognizers into deciders, using the same construction as for TM Acceptance $\leq$ Halting

Thm. Suppose there is a decider H for Halting. Then for every language L, the existence of recognizer M implies the existence of a decider S.

Pf. Idea: Can safely simulate M on w

Decider S for TM Acceptance

On input $\langle M, w \rangle$:

Run H on $\langle M, w \rangle$

If H accepts,

Run M on w

Accept if M accepts

Reject if M rejects

Else, reject

Correctness:

- Always halts
- If $w \in L$, then M accepts and S accepts
- If $w \notin L$, then M rejects and S rejects, or M does not halt and S rejects